

ADC Sensor Log Processor Development Logbook

Engineering domain understanding

An ADC (Analogue-to-Digital Converter) turns a continuous analogue voltage into a digital number the software can actually work with. Sensors measure the real world in analogue; microcontrollers only understand digital, so something has to do this conversion.

This dataset comes from a 4-channel, 12-bit ADC. 12 bits gives 4096 possible levels, 0 to 4095, so `raw_value` on its own isn't a voltage, it's just where the reading landed on that scale. To get the actual voltage I used:

$$\text{voltage} = (\text{raw_value} / 4095.0) * 3.3$$

3.3 V is the reference voltage. I used 4095.0 rather than 4095 to force floating-point division, since `raw_value / 4095` in integer arithmetic would truncate and quietly give wrong voltages for almost every reading. I actually caught this the hard way, my first version used plain 4095 and every voltage came back as either 0.0 or 3.3 with nothing in between, which was obviously broken.

The four channels are interleaved: channel 0, 1, 2, 3, then it repeats. 1000 samples per channel, 4000 records total.

`status_flags` packs two boolean states into one byte instead of using separate variables, which is normal for embedded systems where memory is tight. Bit 0 is a sensor fault, bit 1 is out-of-range. I tested both with bitwise AND.

`sequence_number` should increase by exactly one each record. If the next number is more than one greater than the last, records were dropped somewhere in the stream.

Design rationale

I split the project across five files, mostly so I could keep the ADC logic separate from file I/O and not have one giant `main.c` I'd hate reading back in a week.

`main.c` only handles the flow: read the argument, call the file reader, run the analysis, write the two reports, free memory, exit. Nothing analytical lives there.

`adc.h` holds the constants, structs, and function prototypes, the file header struct, the binary record struct, the processed sample struct, channel stats, and sequence gaps.

adc.c does the actual ADC work, converting raw values to voltage, computing per-channel stats, counting faults, finding sequence gaps.

io.c/io.h handle reading the binary file and writing the reports. This felt like it belonged separately from adc.c since it's about files, not sensor math.

stats.c/stats.h has mean, min, max, and standard deviation. Small, boring, and reusable, which is exactly what I wanted from it.

The binary header and record structs are packed. Without that, the compiler inserts padding bytes to keep fields aligned, and fread() ends up reading garbage because the struct no longer matches the 16-byte layout on disk. I learned this the annoying way in a previous module and made sure not to repeat it here.

Pointer and memory management

Since record_count comes from the file header at runtime, I can't just declare a fixed array, so the sample array is heap-allocated:

```
samples = malloc(header.record_count * sizeof(ADCSample));
```

If malloc() returns NULL the program stops with an error rather than carrying on and segfaulting later, which is a much worse debugging experience.

read_adc_file() takes ADCSample **samples because it allocates memory inside the function and needs to update the pointer that main.c is holding. I also used direct pointer arithmetic in a couple of places, for example:

```
ADCSample *current = *samples + i;
```

which is functionally identical to indexing but makes the pointer movement explicit. In the analysis functions I used const ADCSample *current = samples + i; instead, since those functions only read the data and I wanted the compiler to stop me if I accidentally tried to write through it.

free(samples) runs before exit, and the temporary per-channel voltage arrays are freed right after their stats are computed, not left until the end.

Testing and debugging

Test 1, correct binary file. Ran adc_sensor_log.bin expecting both results.txt and fault_report.txt to appear. Got:

Analysis complete. Summary written to results.txt Fault report written to fault_report.txt

Passed.

Test 2, missing input file. Pointed it at `missing_file.bin`, expecting a clean error instead of a crash. Got "Error: could not open input file." Passed.

Test 3, no argument. Ran the program with nothing after it, expecting a usage message. Got one. Passed.

Test 4, record counts. Expected 4000 records total, 1000 per channel. `results.txt` confirmed all four channels at 1000 each. Passed.

Test 5, fault detection. Expected channel 1 to show overvoltage, channel 2 undervoltage, channel 3 status-flag faults. Got 6 overvoltage on channel 1, 7 undervoltage on channel 2, and on channel 3, 7 sensor faults plus 2 out-of-range, 22 fault entries total. Passed.

Test 6, sequence gaps. Found two: previous 798 to next 801 (2 missing), and previous 2497 to next 2500 (2 missing). Passed.

Bugs and fixes

Bug 1: input file wouldn't open. In CLion this printed "Error: could not open input file." even though the file was right there in my project folder. Took me a minute to realize CLion runs the executable from `cmake-build-debug`, not the project root, so the program was looking in the wrong directory entirely. Fixed by setting the working directory in the run configuration to the project folder. Confirmed by rerunning, `results.txt` appeared as expected.

Bug 2: unresolved symbols. CLion flagged `ADCHeader`, `ADCSample`, and `uint32_t` as unresolved. Turned out `adc.h` was missing `#include <stdint.h>`, and I'd also forgotten `io.h` needed to include "adc.h". Added both, reloaded CMake, and the errors cleared.

Extension: fault report file

I added `write_fault_report_file()` to generate `fault_report.txt`, which lists every individual faulted record with its record number, timestamp, channel, raw value, converted voltage, sequence number, and fault type.

`results.txt` only gives counts. `fault_report.txt` gives the actual trail, which channel, which record, what kind of fault, so you can go back and pin down exactly when something went wrong instead of just knowing that it did. A record can show up more than once here if it has multiple fault types at once, for example `status_flags = 0x03` means both bit 0 and bit 1 are set, so that record gets logged under both fault types.

Final output summary

The program outputs results.txt (header info, per-channel stats, fault counts, sequence gaps, and my interpretation of what they mean) and fault_report.txt (every fault event, individually).

Final numbers: 4000 records processed, 1000 per channel, 6 overvoltage events on channel 1, 7 undervoltage on channel 2, 7 sensor faults and 2 out-of-range events on channel 3, 2 sequence gaps, 22 fault report entries in total.